

B28: SQL SuperHero Program : Session 41

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

2 May 2025 at 07:16

BULK COLLECT

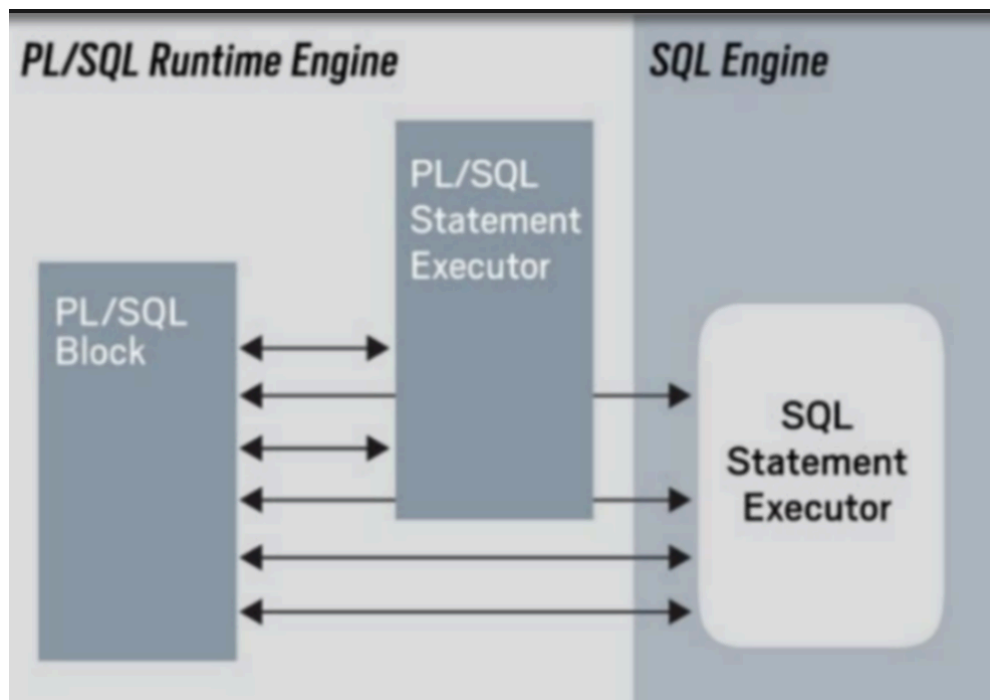
BULK COLLECT: SELECT statements that retrieve multiple rows with a single fetch, improving the speed of data retrieval

FORALL: INSERTs, UPDATEs, and DELETEs that use collections to change multiple rows of data very quickly

Why we need this:

Performance is the answer.

When the PL/SQL runtime engine encounters a SQL statement, it stops and passes the SQL statement over to the SQL engine. The SQL engine executes the SQL statement and returns information back to the PL/SQL engine. This transfer of control is called a context switch, and each one of these switches incurs overhead that slows down the overall performance of your programs.



Let's look at a concrete example to explore context switches more thoroughly and identify the reason that FORALL and BULK COLLECT can have such a dramatic impact on performance.

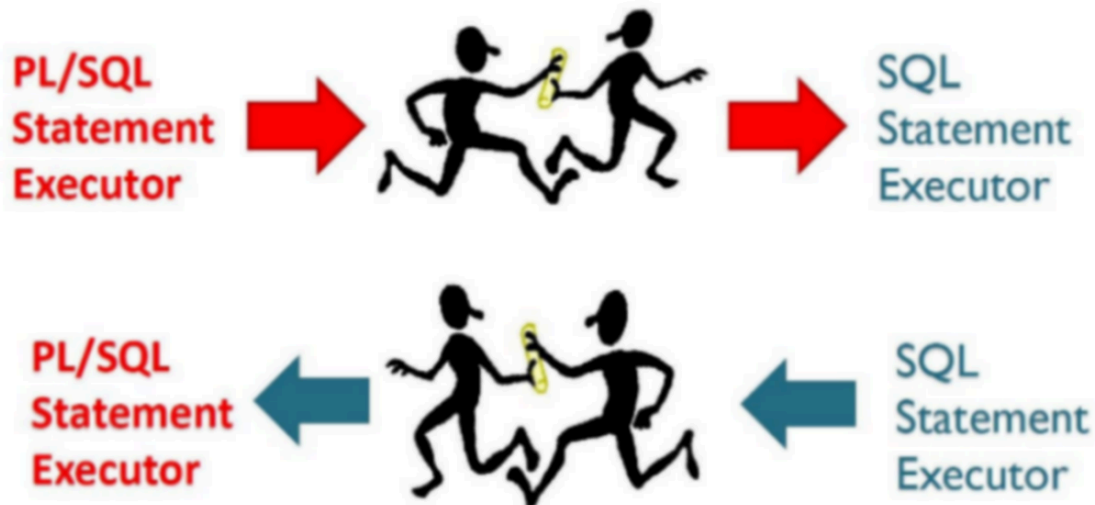
- The keywords BULK COLLECT tell the SQL engine to bulk-bind output collections before returning them to the PL/SQL engine.
- The keyword FORALL instructs the PL/SQL engine to bulk-bind input collections before sending them to the SQL engine. Although the FORALL statement contains an iteration scheme, it is not a FOR loop. Its syntax follows:

```
FORALL index IN lower_bound..upper_bound  
  sql_statement;
```

What is Context Switching?

Whenever you write a PL/SQL block and execute it, the PL/SQL runtime engine starts processing it line by line. This engine processes all the PL/SQL statements by itself but it passes all the SQL statements which you have coded into that PL/SQL block to the SQL runtime engine. Those SQL statements will then get processed separately by the SQL engine. Once it is done processing them the SQL engine then returns the result back to the PL/SQL engine. So that a combined result can be produced by the latter. This to and fro hopping of control is called context switching.

CONTEXT SWITCHING



For Loop

The numeric FOR LOOP executes the body of a loop once for every integer value in a specified range.

Any variable

```
1 Begin
2   For I in 1 .. 5 Loop
3     dbms_output.put_line(i);
4   End Loop;
5 End;
```

for 1 to 5 looping

Statement processed.

```
1
2
3
4
5
```

-Printed here

See the above simple example,

For Loop , means loop from lower bound to upper bound. In example it was saying loop 1 to 5 , so loop will run 5 times 1 to 5 and print value.

```

1 Begin
2   For X in (select * from hr.employees where department_id = 90) Loop
3     dbms_output.put_line(x.first_name);
4   End Loop;
5 End;

```

Statement processed.
Steven
Neena
Lex

Handwritten notes:
All the records from here will be in loop
x is variable
processed all records

see the example, here we wrote

For X in (select * from hr.employees where department_id = 90) Loop

It means all records given by inner query, will be going through a loop. we took a variable 'X' that variable will be used to access data, that is coming from an inner query.

so in next line, when we printing, we are using X.first_name

the loop will run number of records it has from inner query. Loop is running one by one.

Requirement: write a procedure that accepts a department ID and a salary percentage increase and gives everyone in that department a raise by the specified percentage

Method 1: For Loop --> Row By Row processing of cursor

```

PROCEDURE increase_salary (
  department_id_in IN employees.department_id%TYPE,
  increase_pct_in IN NUMBER)
IS
BEGIN
  FOR employee_rec
  IN (SELECT employee_id
      FROM employees
      WHERE department_id = department_id_in)
  LOOP
    UPDATE employees emp
    SET emp.salary = emp.salary + emp.salary * increase_pct_in
    WHERE emp.employee_id = employee_rec.employee_id;
  END LOOP;
END increase_salary;

```

Suppose there are 100 employees in department 15. When I execute this block,

```

BEGIN
  increase_salary (15, .10);
END;

```

--> the PL/SQL engine will "switch" over to the SQL engine 100 times, once for each row being updated. row-by-row switching like this as "slow-by-slow processing," and it is definitely something to be avoided.

Take another look at the increase_salary procedure. In such a simple scenario, a cursor FOR loop is not needed at all. if we do a direct 1 update statement, which update the salary. simple update. 1 statement. 1 context switch

The bulk processing features of PL/SQL are designed specifically to reduce the number of context switches required to communicate from the PL/SQL engine to the SQL engine.

Use the BULK COLLECT clause to fetch multiple rows into one or more collections with a single context switch.



Simple Loop Vs Bulk Collect

select count(*) from hr.employees; -- 107

I have a table employee where I want to insert all data from hr.employees table.

create table employees as select * from hr.employees where 1 = 2; --> a new table is created with same structure and 0 records

Begin

For I In (select * from hr.employees) Loop

**Insert Into employees Values (I.EMPLOYEE_ID,I.FIRST_NAME, I.LAST_NAME, I.EMAIL,
I.PHONE_NUMBER, I.HIRE_DATE, I.JOB_ID,I.SALARY, I.COMMISSION_PCT,I.MANAGER_ID,
I.DEPARTMENT_ID);**

Dbms_output.Put_Line('Row Number '||sql%rowcount);

End Loop;

End;

```

7  Begin
8      For I In (select * from hr.employees) Loop
9          Insert Into employees Values (I.EMPLOYEE_ID,I.FIRST_NAME, I.LAST_NAME, I.EMAIL,
10             I.PHONE_NUMBER, I.HIRE_DATE, I.JOB_ID,I.SALARY,
11             I.COMMISSION_PCT,I.MANAGER_ID,I.DEPARTMENT_ID);
12          Dbms_output.Put_Line('Row Number '||sql%rowcount);
13      End Loop;
14  End;

```

→ Run everytime

Statement processed.

Row Number 1
Row Number 1
Row Number 1
Row Number 1
Row Number 1
Row Number 1
Row Number 1
Row Number 1
Row Number 1
Row Number 1

For every row it is printed.

We can see that it run for every row and printed, so the number of records in my table loop is running such number of time. here it run 107 times.

CREATE TABLE t3 AS SELECT * FROM hr.employees WHERE 1 = 2;

declare

TYPE t_emp_data IS TABLE OF hr.employees%ROWTYPE;

t_emp_data_aa t_emp_data;

BEGIN

**SELECT * BULK COLLECT INTO t_emp_data_aa
FROM hr.employees;**

```
FORALL x in t_emp_data_aa.First..t_emp_data_aa.Last  
INSERT INTO t3 VALUES t_emp_data_aa(x);  
dbms_output.put_line('Row Number '||sql%rowcount);  
END;
```

```
3 declare  
4  
5 TYPE t_emp_data IS TABLE OF hr.employees%ROWTYPE;  
6 t_emp_data_aa t_emp_data;  
7  
8 BEGIN  
9     SELECT * BULK COLLECT INTO t_emp_data_aa  
10    FROM hr.employees;  
11  
12    FORALL x in t_emp_data_aa.First..t_emp_data_aa.Last  
13    INSERT INTO t3 VALUES t_emp_data_aa(x) ;  
14    dbms_output.put_line('Row Number '||sql%rowcount);  
15 END;
```

Statement processed.
Row Number 107

Just 1 shot